

Q1.

[5 marks]

You are tasked with writing an assembly language program for a fitness tracker that monitors the number of steps taken and calories burned over two different weeks. The system should perform necessary calculations to adjust and analyze the data based on user input. The system should work with signed and unsigned values and perform various arithmetic and logical operations.

1. Start by defining an unsigned 16-bit integer (caloriesBurned) that stores the total calories burned during the week. Define two signed 8-bit integers (stepsWeek1 and stepsWeek2) to represent the number of steps taken in the first and second weeks.
2. Load the values of stepsWeek1 and stepsWeek2 into registers, ensuring the 8-bit signed integers are extended to 32-bit registers using sign-extension. Load the caloriesBurned value and zero-extend it to a 32-bit register.
3. Calculate the total steps by adding the values of stepsWeek1 and stepsWeek2.
4. After calculating total steps, decrement the number of steps taken in Week 2 (stepsWeek2) by a constant value (e.g., 5).
5. Swap the values of stepsWeek1 and stepsWeek2 in the registers to simulate a data error.

Page 1 of 2

6. Subtract the adjusted steps in Week 2 from the caloriesBurned to determine if the calories burned are sufficient for the steps taken.
7. If the result indicates a deficit, negate it and store the final value in a register.
8. Output the final results: the total steps taken, adjusted steps for Week 2, and the result indicating whether there is a caloric deficit.

```
INCLUDE Irvine32.inc
```

```
.data
```

```
caloriesBurned WORD 20
```

```
stepsWeek1 SBYTE 20
```

```
stepsWeek2 SBYTE 30
```

```
.code
```

```
main PROC
```

```
    mov eax, 0
```

```
    mov ebx, 0
```

```
    mov edx, 0
```

```
    movsx eax, stepsWeek1
```

```
    movsx ebx, stepsWeek2
```

```
    movzx edx, caloriesBurned
```

```
    add eax, ebx ;sum of week1 step sand week2 steps
```

```
    sub ebx, 5 ;decrementing steps of week 2 by 5
```

```

        ;xchg stepsWeek1, stepsWeek2 ;error cannot xchg form mem to mem

        sub edx, ebx
        js deficit
        jmp end1

deficit:
        neg edx
        jmp end1

end1:
        call WriteInt ; total steps
        call crlf
        mov eax, ebx
        call WriteInt ; steps 2 after adjust
        call crlf
        mov eax, edx
        call WriteInt ; deficit or not
        call crlf

        call DumpRegs

        exit
main ENDP
END main

```

Q2.

[5 marks]

You are assigned to develop a simple billing system for a small store that sells a single item. At the end of the day, the store owner wants to record the quantity sold and calculate the total cost. The system should prompt the user to input the quantity sold for the item, and then calculate the total cost based on a predefined price per item. The program should also check if the total cost exceeds a certain threshold and, if so, apply a discount. The total cost and any applicable discounts should then be displayed to the user. Use Irvine32 assembly language to implement this billing system implementing the solution with Arithmetic Instructions (add, mul, sub) and Conditional Jumps (jmp, cmp).

```

INCLUDE Irvine32.inc

.data
quantityPrompt BYTE "Enter quantity sold: ", 0
price DWORD 70 ; predefined 70 PKR price
discount DWORD 50 ;50 PKR discount
displayTotalCost BYTE "Your total cost: PKR ", 0
displayDiscountApplied BYTE "Discount of PKR 50 has been applied", 0
displayNoDiscount BYTE "No discount applicable",0

.code

```

```

main PROC
    mov eax, 0
    mov ebx, 0
    mov ecx, 0
    mov edx, 0

    mov edx, OFFSET quantityPrompt
    call WriteString
    call readDec

    MUL price ; eax now stores total cost
    cmp eax, 1000 ;if total cost above 1000
    jg addDiscount
    jmp noDiscount

addDiscount:
    sub eax, discount
    mov edx, OFFSET displayDiscountApplied
    call WriteString
    call crlf
    jmp display

noDiscount:
    mov edx, OFFSET displayNoDiscount
    call WriteString
    call crlf
    jmp display

display:
    mov edx, OFFSET displayTotalCost
    call WriteString
    call WriteDec

    exit
main ENDP
END main

```

Q3.

[5 marks]

You are tasked with developing a Snack Inventory System for a convenience store that sells five types of snacks. The system will record sales for two weeks using indirect operands to access the arrays. After inputting sales data for each week, it will calculate the total sales for each snack type by summing the sales from both weeks. The program should display the total sales count for each snack type without using loops, relying on indirect operands for array access.

```
INCLUDE Irvine32.inc
```

```
.data
week1Sales DWORD 5 DUP (?)
week2Sales DWORD 5 DUP (?)
sales1Prompt BYTE "Enter sales of all 5 products for week 1: ", 0
sales2Prompt BYTE "Enter sales of all 5 products for week 2: ", 0
totalSalesPrompt BYTE "Total sales combined for all products: ", 0
```

```
.code
main PROC
    mov eax, 0
    mov ebx, 0
    mov ecx, 0
    mov edx, 0
    mov esi, 0
    mov esi, OFFSET week1Sales
    mov edi, 0
    mov edi, OFFSET week2Sales

    mov edx, OFFSET sales1Prompt
    call WriteString
    call ReadDec
    mov [esi], eax
    add esi, 4
    call ReadDec
    mov [esi], eax
    add esi, 4
    call ReadDec
    mov [esi], eax
    add esi, 4
    call ReadDec
    mov [esi], eax
    add esi, 4
    call ReadDec
    mov [esi], eax
    add esi, 4

    mov edx, OFFSET sales2Prompt
    call WriteString
    call ReadDec
    mov [edi], eax
    add edi, 4
    call ReadDec
    mov [edi], eax
    add edi, 4
    call ReadDec
    mov [edi], eax
    add edi, 4
    call ReadDec
    mov [edi], eax
    add edi, 4
```

```
call ReadDec
mov [edi], eax
add edi, 4
```

```
mov esi, 0
mov edi, 0
mov esi, OFFSET week1Sales
mov edi, OFFSET week2Sales
```

```
mov edx, OFFSET totalSalesPrompt
call WriteString
call crlf
mov eax, 0
mov eax, [esi]
add eax, [edi]
call WriteDec
call crlf
add esi, 4
add edi, 4
```

```
mov eax, 0
mov eax, [esi]
add eax, [edi]
call WriteDec
call crlf
add esi, 4
add edi, 4
```

```
mov eax, 0
mov eax, [esi]
add eax, [edi]
call WriteDec
call crlf
add esi, 4
add edi, 4
```

```
mov eax, 0
mov eax, [esi]
add eax, [edi]
call WriteDec
call crlf
add esi, 4
add edi, 4
```

```
mov eax, 0
mov eax, [esi]
add eax, [edi]
call WriteDec
call crlf
add esi, 4
add edi, 4
```

```

        exit
main ENDP
END main

```

Q4.

Write an assembly program to construct the following pattern.

```

      6
    6 6 6
  6 6 6 6 6
6 6 6 6 6 6 6
  8 8 8
  8 8 8
  8 8 8
  8 8 8

```

```

INCLUDE Irvine32.inc

.data
num DWORD 6
space BYTE " ", 0

.code
main PROC
    mov eax, 0
    mov ebx, 0
    mov ecx, 0
    mov edx, 0

    mov ecx, 4
    mov eax, num
    mov esi, 8 ; space counter before digits
    mov ebx, 1 ; digit counter

L1:
    push ecx
    mov ecx, esi
    mov edx, OFFSET space
    L2:
        call WriteString
        cmp ecx, 0
        je esiover
        loop L2

esiover:
    mov ecx, 0
    mov ecx, ebx
    mov edx, OFFSET space
    L3:

```

```

        call WriteDec
        call WriteString
        cmp ecx, 0
        je ebxover
        loop L3

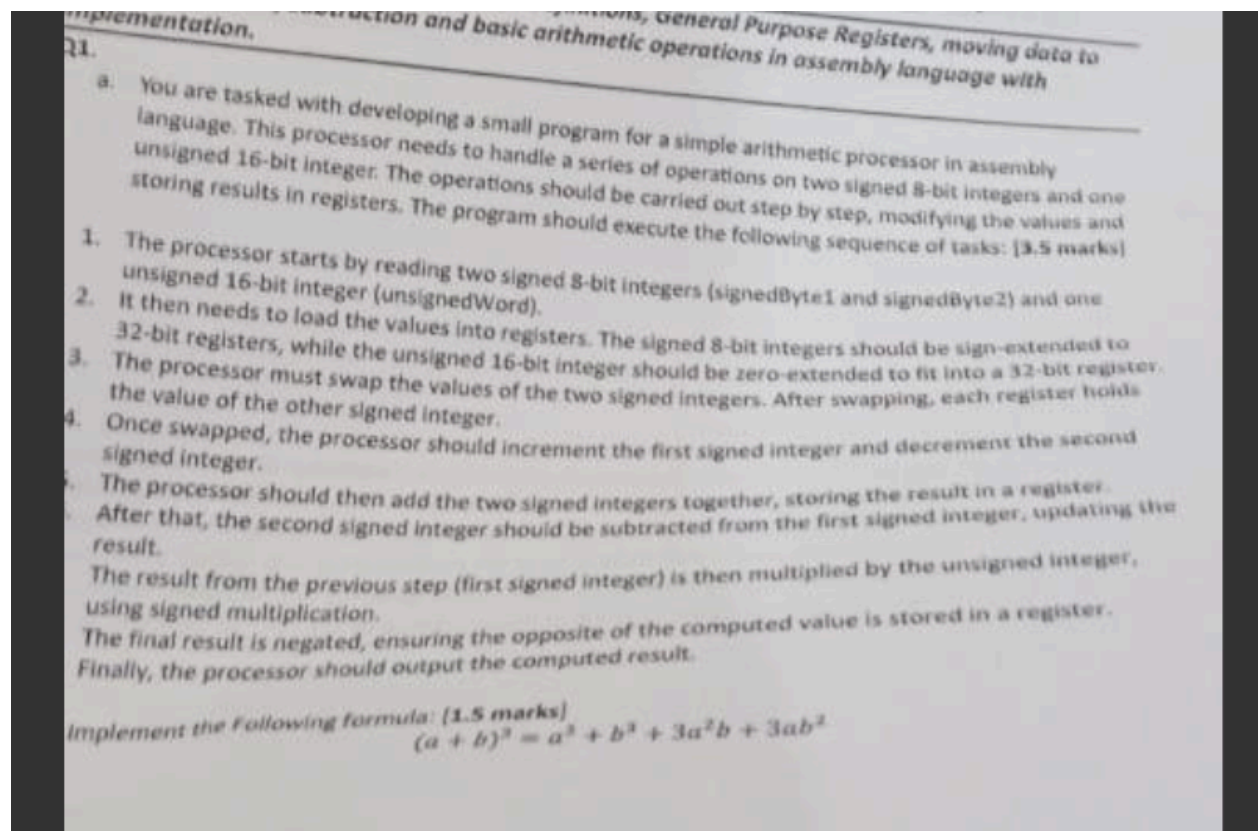
ebxover:
    add ebx, 2
    sub esi, 2
    call crlf
    pop ecx
    cmp ecx, 0
    je nextpattern
    loop L1

nextpattern:
    mov ecx, 0
    mov ecx, 4
    mov edx, OFFSET space
L5:
    push ecx
    mov ecx, 6
    mov edx, OFFSET space
L6:
    call WriteString
    cmp ecx, 0
    je spaceover
    loop L6

spaceover:
    mov eax, 8
    call WriteDec
    call WriteString
    call WriteDec
    call WriteString
    call WriteDec
    call crlf
    pop ecx
    cmp ecx, 0
    je end1
    loop L5

end1:
    exit
main ENDP
END main

```



```
INCLUDE Irvine32.inc

.data
signedByte1 SBYTE ?
signedByte2 SBYTE ?
unsignedWord WORD ?

.code
main PROC
    mov eax, 0

    call readInt
    mov signedByte1, al
    call readInt
    mov signedByte2, al
    call readInt
    mov unsignedWord, ax

    movsx eax, signedByte1
    movsx ebx, signedByte2
    movzx edx, unsignedWord

    xchg eax, ebx
    inc ebx ;first
```



```

    dec eax ;second

    mov ecx, eax
    add ecx, ebx ;add

    sub ebx, eax ;sub second from first

    mov eax, ebx ;overiding second
    mul edx ;mul first w word

    neg eax

    ;mul
    call WriteInt
    call crlf
    ; sum
    mov eax, ecx
    call WriteInt
    call crlf

    exit
main ENDP
END main

```

FORMULA QS:

```
INCLUDE Irvine32.inc
```

```

.data
a DWORD ?
b DWORD ?
count DWORD 3

```

```

.code
main PROC
    mov ebx, 3    ; a = 3
    mov ecx, 3    ; b = 3

    mov a, ebx
    mov b, ecx

    ; Calculate (a + b)^3
    mov eax, ebx    ; eax = a
    add eax, ecx    ; eax = a + b
    mov ebx, eax
    mov edx, eax    ; edx = a + b
    mul edx
    mul ebx
    call WriteInt    ; print (a + b)^3
    call crlf

```

```

mov ebx, 0
; Calculate a^3 + b^3 + 3a^2b + 3ab^2
; Step 1: a^3
mov eax, a
mul eax
mul a
add ebx, eax

; Step 2: b^3
mov eax, b
mul eax
mul b
add ebx, eax

; Step 3: 3a^2b
mov eax, a
mul a
mul b
mul count
add ebx, eax

; Step 4: 3ab^2
mov eax, b
mul b
mul a
mul count
add ebx, eax

; Output the result
mov eax, ebx
call writeint

exit
main ENDP
END main

```

CLO # 2:

[5 marks]**Q2.**

Consider yourself a customer creating an account on an e-commerce website. Write an assembly program that inputs the username and card info (**your roll number**) and assigns a random generated account number to the user. The program should then display the assigned account number to the user in a message box. What is the output in Message Box? Explain.

```
INCLUDE Irvine32.inc

.data
str1 BYTE "Enter your username: ", 0
str2 BYTE "Enter your card number (roll number): ", 0
str3 BYTE "Your random account number is: ", 0
username BYTE 30 DUP(0)
cardNumber BYTE 10 DUP(0)
randomAccountNumber DWORD ?
msgBoxText BYTE 50 DUP(0)

.code
main PROC
    mov edx, OFFSET str1
    call WriteString
    mov edx, OFFSET username
    mov ecx, 30
    call ReadString

    mov edx, OFFSET str2
    call WriteString
    mov edx, OFFSET cardNumber
    mov ecx, 10
    call ReadString

    call randomize
    mov ecx, 100
    call randomRange
    call WriteDec ; to see number generated
    ;mov eax, 65 to check msgbox giving ascii value
    mov randomAccountNumber, eax

    mov msgBoxText, al

    mov edx, OFFSET msgBoxText
    mov ebx, OFFSET str3
    call MsgBox

    exit
main ENDP
```

END main

a. Implement the following high-level mathematical equations into assembly language *using x86 general purpose registers*.

1. $EAX = 98 - F74h + 74o + 82 - 101010b$
2. $EAX = Val1 - Val2 + 456h + Val3$

Val1	WORD	25h
Val2	DWORD	36o
Val3	BYTE	20d

```
INCLUDE Irvine32.inc
.DATA
.CODE
main PROC
mov eax, 89 ; EAX = 89
add eax, 75Fh ; EAX = EAX + 75Fh
sub eax, 46o ; EAX = EAX - 46o
sub eax, 28 ; EAX = EAX - 28
add eax, 1101b ; EAX = EAX + 1101b
call DumpRegs
exit
main ENDP
END main
```

```
INCLUDE Irvine32.inc
.DATA
Val1 DWORD 25h ;
Val2 BYTE 36o ;
Val3 WORD 20d ;
.CODE
main PROC
mov eax, Val1 ;
movzx ebx, Val2 ;
add eax, ebx ;
sub eax, 654h ;
movzx ecx, Val3 ;
add eax, ecx ;
call DumpRegs ;
exit
main ENDP
END main
```

- b. Declare a 16-bit signed integer `val1` and initialize it with the smallest possible decimal value. Display its value on the console window. Increment its value by 3 units and again display the result on the console window. There should be a delay of 1.5s between two outputs. *[Display results 3 times]*.**

```

INCLUDE Irvine32.inc

.data
val1 SWORD -32768

.code
main PROC
    movsx eax, val1
    call WriteInt
    call crlf
    push eax ;saving for delay
    mov ecx, 3
L1:
    mov eax, 1500
    call delay
    pop eax ;popping
    add eax, 3
    call WriteInt
    call crlf
    push eax ;saving again for next loop delay
    loop L1

    exit
main ENDP
END main

```

Question # 2

[3 Points]

Use following array declarations:

```

arrayW DW 50,20,90,101,450
arrayB DB 10,24,67,90,100

```

Add even index of arrayW with odd index of arrayB and store that value in the array of 32-bit of your own name. Use of Scale Factor is mandatory

```

INCLUDE Irvine32.inc

.data
arrW DWORD 50,20,90,101,450
arrB DWORD 10,24,67,90,100
arrC DWORD 2 DUP(?)

.code
main PROC

    mov esi, 0
    mov edi, 1
    mov eax, arrW[esi * TYPE arrW]
    mov ebx, arrB[edi * TYPE arrB]
    add eax, ebx

```

```

        mov arrC[0], eax

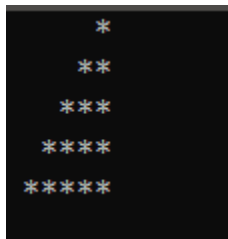
        mov esi, 2
        mov edi, 3
        mov eax, arrW[esi * TYPE arrW]
        mov ebx, arrB[edi * TYPE arrB]
        add eax, ebx
        mov arrC[1], eax

        mov esi, 0
        mov ecx, 2

print_loop:
        mov eax, arrC[esi]
        call WriteDec
        call Crlf
        inc esi
        loop print_loop

        exit
main ENDP
END main

```



```

INCLUDE Irvine32.inc
.data
cols DWORD 5
space BYTE " ", 0
.code
main proc
        mov eax, 42
        mov ebx, cols
        mov edx, OFFSET space

        mov esi, 5; space counter
        mov edi, 1 ;star counter
        mov ecx, ebx

L1:
        push ecx
        mov ecx, esi
        L2:
                call WriteString
                cmp ecx, 0

```

```

        je esiover
        loop L2

esiover:
    mov ecx, edi
    L3:
        call WriteChar
        loop L3
    call crlf
    pop ecx
    dec esi
    inc edi
    loop L1

exit
main endp
END main

```

Fibonacci

```

INCLUDE Irvine32.inc
.code
main PROC
    mov eax, 1
    mov ebx, 1 ; cuz first 2 terms same
    mov ecx, 10
    call writeDec
    call crlf
    call writeDec
    call crlf
    L1:
    mov edx, eax
    add eax, ebx ; adds prev num
    mov ebx, edx ; storing prev num for further use
    call writeDec ; only shows eax
    call crlf
    loop L1
    exit
main ENDP
END main

```

Bubble Sort:

```

INCLUDE Irvine32.inc
.data
val1 BYTE 40, 10, 30, 20, 50
count DWORD 0
.code
main PROC
    mov ecx, 4
    L1:
    mov count, ecx

```

```

mov esi, 0
L2:
mov al, [val1 + esi]
mov bl, [val1 + esi + 1]
cmp al, bl
jle noswap
mov [val1 + esi + 1], al

```

Name: Nabira Khan
Roll Number: 23K-0914

```

mov [val1 + esi], bl
noswap:
add esi, 1
cmp esi, count
jl L2
mov ecx, count
loop L1
mov esi, 0
mov ecx, LENGTHOF val1
L3:
mov al, [val1 + esi]
movzx eax, al
call WriteDec
call Crlf
add esi, 1
loop L3
exit
main ENDP

```

Reversing array

```

INCLUDE Irvine32.inc
.data
val1 BYTE 10, 20, 30, 40, 50
len BYTE LENGTHOF val1
reverse BYTE LENGTHOF val1 DUP(0)
.code

```

Name: Nabira Khan
Roll Number: 23K-0914

```

main PROC
movzx ecx, len
mov esi, OFFSET val1
mov edi, OFFSET reverse
L1:
mov al, [esi + ecx - 1] ; zero based indexing hence - 1
mov [edi], al
add edi, TYPE reverse
loop L1

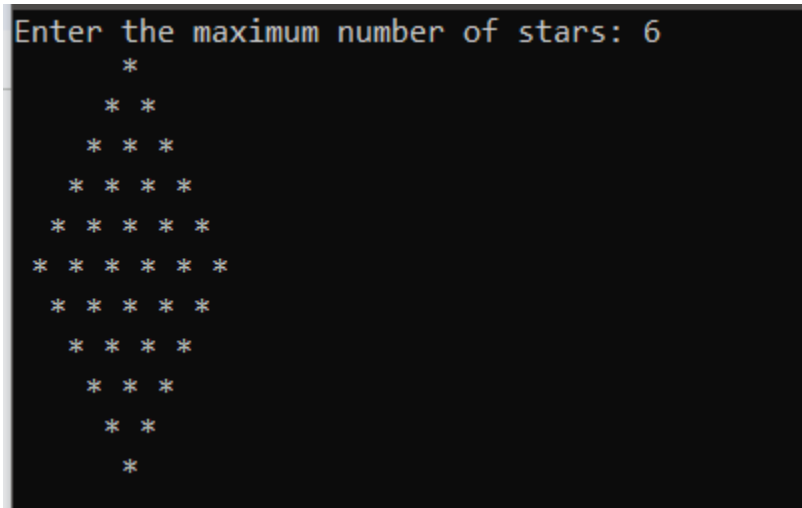
```



```

mov ecx, 5
mov esi, OFFSET reverse
L2: ; printing loop
mov eax, 0
mov al, [esi]
call WriteDec
call Crlf
add esi, TYPE reverse
loop L2
exit
main ENDP
END main

```



```

INCLUDE Irvine32.inc

.data
    star BYTE "*", 0
    space BYTE " ", 0
    n DWORD ?
    input BYTE "Enter the maximum number of stars: ", 0

.code
main PROC
    mov edx, OFFSET input
    call WriteString
    call readDec
    mov n, eax

    mov ecx, n
    mov eax, 1

L1:
    push ecx

```

```

    mov ebx, ecx
    mov ecx, ebx

L2:
    mov edx, OFFSET space
    call WriteString
    loop L2

    mov ecx, eax

L3:
    mov edx, OFFSET star
    call WriteString
    mov edx, OFFSET space
    call WriteString
    loop L3

    inc eax

call crlf
pop ecx

loop L1

dec n
mov ecx, n
mov eax, 2

L4:

    mov ebx, ecx
    push ecx
    mov ecx, eax

L5:
    mov edx, OFFSET space
    call WriteString
    loop L5

    inc eax
    mov ecx, ebx

L6:
    mov edx, OFFSET star
    call WriteString
    mov edx, OFFSET space
    call WriteString
    loop L6

call crlf
pop ecx

```

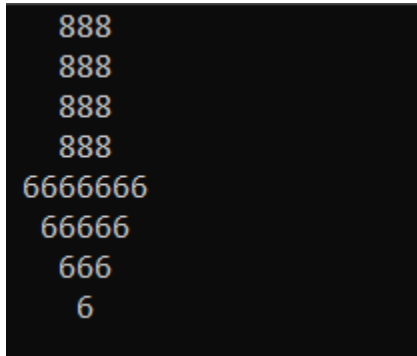
```

        loop L4

        exit
main ENDP

END main

```



```

INCLUDE Irvine32.inc

.data
    space BYTE " ", 0
    numberLOOP BYTE ?

.code
main PROC

    ; print tail
    mov ecx, 4

L4:
    push ecx
    mov ecx, 3

    L5:
        mov edx, OFFSET space
        call WriteString
    loop L5

    mov ecx, 3

    L6:
        mov eax, 8
        call WriteDec
    loop L6

    call crlf
    pop ecx

    loop L4

```

```

; print head

mov ecx, 4
mov ebx, 1
mov numberLOOP, 7

L1:
    push ecx
    mov ecx, ebx

    L2:
        mov edx, OFFSET space
        call WriteString
    loop L2

    inc ebx

    movzx ecx, numberLOOP

    L3:
        mov eax, 6
        call WriteDec
    loop L3

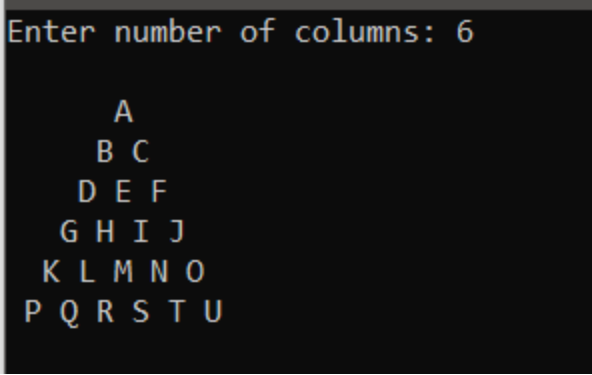
    sub numberLOOP, 2

    call crlf
    pop ecx
loop L1

call Dumpregs

exit
main ENDP
END main

```



```
Enter number of columns: 6
```

```
  A
 B C
D E F
G H I J
K L M N O
P Q R S T U
```

```
INCLUDE Irvine32.inc
```

```
.data
```

```
char BYTE "A", 0
space BYTE " ", 0
input BYTE "Enter number of columns: ", 0
col DWORD ?
```

```
.code
```

```
main PROC
```

```
mov edx, OFFSET input
call WriteString
call ReadDec
mov col, eax
call crlf
```

```
call printPattern
call crlf
exit
main ENDP
```

```
printPattern PROC
```

```
mov ecx, col
mov eax, 1
```

```
L1:
```

```
push ecx
mov ebx, ecx
mov ecx, ebx
```

```
L2:
```

```
mov edx, OFFSET space
call WriteString
loop L2
```

```
mov ecx, eax
```

```

        L3:
            mov edx, OFFSET char
            call WriteString
            inc char
            mov edx, OFFSET space
            call WriteString
        loop L3

        inc eax

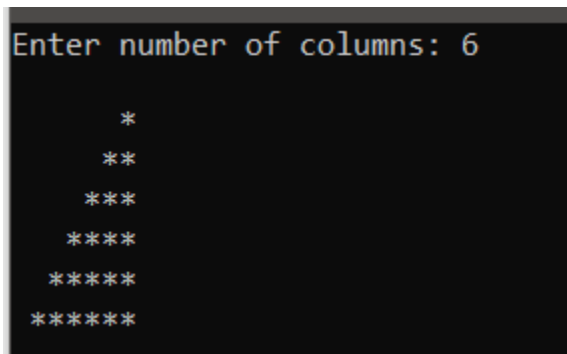
        call crlf
        pop ecx

        loop L1

    ret
printPattern ENDP

END main

```



```

INCLUDE Irvine32.inc

.data
    star BYTE "*", 0
    space BYTE " ", 0
    input BYTE "Enter number of columns: ", 0
    col DWORD ?

.code
    main PROC

        mov edx, OFFSET input
        call WriteString
        call ReadDec
        mov col, eax
        call crlf

        call printPattern
        call crlf
    
```

```

exit
main ENDP

printPattern PROC

    mov ecx, col
    mov eax, 1

L1:
    push ecx
    mov ebx, ecx
    mov ecx, ebx

    L2:
        mov edx, OFFSET space
        call WriteString
    loop L2

    mov ecx, eax

    L3:
        mov edx, OFFSET star
        call WriteString
    loop L3

    inc eax

    call crlf
    pop ecx

    loop L1

ret
printPattern ENDP

```

END main



```

include irvine32.inc
.data

```

```
star BYTE "* ",0
space BYTE " ",0
OuterSpace DWORD 4
innerSpace DWORD 2
rows DWORD 4
```

```
OutwardSpace DWORD 2
inwardSpace DWORD 6
```

```
downRows DWORD 3
```

```
.code
```

```
main PROC
```

```
mov ecx,6
```

```
Lfirst:
```

```
    mov edx, OFFSET Space
```

```
    call writeString
```

```
LOOP Lfirst
```

```
mov edx, OFFSET star
```

```
call writeString
```

```
call crlf
```

```
mov ecx, 4
```

```
L1:
```

```
    ;outer
```

```
    mov ecx, OuterSpace
```

```
    L2:
```

```
        mov edx, OFFSET space
```

```
        call writeString
```

```
    LOOP L2
```

```
    ;print
```

```
    mov edx, OFFSET star
```

```
    call writeString
```

```
    ;inner Space
```

```
    mov ecx, innerSpace
```

```
    L3:
```

```
        mov edx, OFFSET space
```

```
        call writeString
```

```
    LOOP L3
```

```
    ;print
```

```
    mov edx, OFFSET star
```

```
    call writeString
```



```

    call crlf

    DEC OuterSpace
    add innerSpace, 2
    mov ecx, rows
    DEC rows

LOOP L1

mov ecx, 3
L4:

    ;outward
    mov ecx, OutwardSpace
    L5:
        mov edx, OFFSET space
        call writeString
        LOOP L5

    ;print
    mov edx, OFFSET star
    call writeString

    ;inward
    mov ecx, inwardSpace
    L6:
        mov edx, OFFSET space
        call writeString
        LOOP L6

        ;print
    mov edx, OFFSET star
    call writeString
    call crlf
    INC OutwardSpace
    sub inwardSpace, 2

    mov ecx, downRows
    dec downRows

    LOOP L4


    mov ecx, 6
Lsecond:
    mov edx, OFFSET Space
    call writeString
    LOOP Lsecond

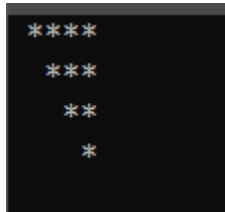
```

```

mov edx, OFFSET star
call writeString

exit
main ENDP
END main

```



```

include irvine32.inc
.data

star BYTE "*",0
space BYTE " ",0

countSp DWORD 1
countSt DWORD 4
rows DWORD 4

.code
main PROC

mov ecx, 4
L1:

    ;space
    mov ecx, countSp
    L2:
        mov edx, OFFSET space
        call writeString
    LOOP L2

    ;stars
    mov ecx, countSt
    L3:
        mov edx, OFFSET star
        call writeString
    LOOP L3
    call crlf
    inc countsp
    dec countst

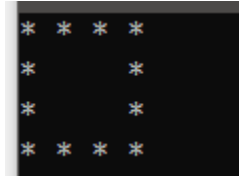
    mov ecx, rows
    dec rows
LOOP L1

```

```

exit
main ENDP
END main

```



```

include irvine32.inc
.data

countMiddle DWORD 2
count DWORD 0
counter DWORD 4
space BYTE " ",0
star BYTE "* ",0

.code
main PROC

mov ecx, 4
L1:

    mov edx, OFFSET star
    call writeString
    LOOP L1
    call crlf

mov ecx, countMiddle

L2:
    mov edx, OFFSET star
    call writeString

    mov ecx, 2
    L3:
        mov edx, OFFSET space
        call writeString

    LOOP L3

    mov edx, OFFSET star
    call writeString

    call crlf

```

```

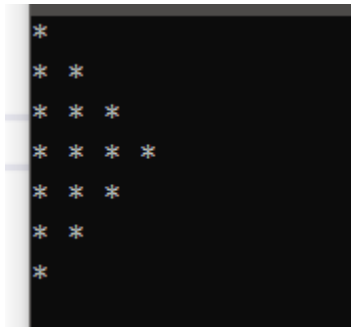
        mov ecx, countMiddle
        DEC countMiddle

LOOP L2

mov ecx, 4
L4:
    mov edx, OFFSET star
    call writeString
    LOOP L4

exit
main ENDP
END main

```



```

Include irvine32.inc
.data
count DWORD 4
star DWORD 1
count1 DWORD 3
star1 DWORD 3
mystr BYTE "* ",0
.code
main PROC
mov ecx, 4

L1:

    mov ecx, star
    mov edx, OFFSET mystr
    L2:
        call writeString
        LOOP L2

    mov ecx, count
    DEC count
    call crlf
    inc star

```

```

        LOOP    L1

mov ecx, count1
L3:

    mov ecx, star1
    mov edx, OFFSET mystr

    L4:
        call writeString
        LOOP L4

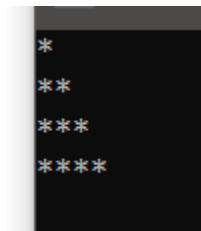
    call crlf
    DEC star1

    mov ecx, count1
        DEC count1

        LOOP L3

    ex:
exit
main ENDP
END main

```



```

Include irvine32.inc
.data

msg BYTE '*',0
loop1 DWORD ?
X BYTE ?
count DWORD 1

.code
main PROC

mov eax, 4 ;number of rows
mov ecx, eax

```

L1:

```
mov loop1 , ecx  
mov ecx, count
```

L2:

```
mov edx , OFFSET msg  
call writestring  
LOOP L2
```

```
inc count  
mov ecx, loop1  
call crlf
```

```
LOOP L1
```

```
exit  
main ENDP  
END main
```